Portuguese Customer Review Score Classification

A01712119 Dongju Mun

Instituto Tecnológico y de Estudios Superiores de Monterrey

***Abstract*—Customer reviews provide information about purchasing experiences, but the numerical rating associated with a written comment is not always explicit. This project develops a multiclass machine-learning pipeline to predict one-to-five-star ratings from Portuguese reviews in the Brazilian E-Commerce Public Dataset by Olist. After data preparation, 40,634 reviews are divided into training, validation, and test subsets. Four classification approaches are examined in sequence: TF-IDF with multinomial logistic regression, a CNN with frozen fastText embeddings, a CNN with learned embeddings, and an ordinal version of the learned-embedding CNN. Two additional regression experiments use a learned-embedding CNN and TF-IDF with Ridge regression. Their rounded validation MAEs are 0.5584 and 0.7795 stars, respectively. The learned Embedding CNN classifier is selected because it combines the highest validation macro-F1 among the four argmax models with a low mean absolute error and greater recognition of intermediate ratings. Its validation accuracy is 68.38%, macro-F1 is 0.4511, and MAE is 0.4803 stars. On 8,127 held-out test reviews, the model obtains approximately 68% accuracy, 0.45 macro-F1, and an MAE of 0.4924 stars. These results support approximate rating estimation while showing substantial limitations in distinguishing two-, three-, and four-star reviews.**

Keywords—natural language processing, Portuguese reviews, TF-IDF, logistic regression, fastText, convolutional neural network, word embeddings, ordinal classification, mean absolute error.

I. Introduction

Online customer reviews contain information about product quality, delivery time, missing items, and the overall purchasing experience. Together with a numerical score, these comments allow other customers and sellers to understand how a transaction was perceived. However, interpreting a large collection of written reviews manually requires time and can produce inconsistent judgments.

Machine learning provides a way to analyze review text and estimate the rating associated with it. In this project, the review title and comment are used as the only predictive inputs. The task contains five ordered categories, from one star to five stars. This order is relevant because predicting four stars for a five-star review represents a smaller rating error than predicting one star.

The analysis begins with a TF-IDF baseline and then examines convolutional models with different word representations and training objectives. The comparison considers both exact classification and the distance between predicted and observed ratings. This provides a more complete interpretation than accuracy alone, particularly when the target classes are imbalanced. Two additional experiments directly predict numerical ratings using an Embedding CNN regressor and TF-IDF with Ridge regression.

II. Objective

The objective of this project is to develop and evaluate a multiclass model that predicts the review score from Portuguese customer comments. The specific goals are to prepare the text data, compare four classification approaches and two additional regression configurations, examine performance for every score, and select a model that provides useful approximate predictions while maintaining recognition across the five categories. The final model is evaluated on a held-out test set and can be reused through a text-input interface without retraining.

III. Background and Theoretical Foundations

A. Text Representation and Word Embeddings

A machine-learning model requires numerical inputs. TF-IDF represents a review through weighted word frequencies, whereas integer vectorization assigns a vocabulary index to each token. An integer index identifies a word; its numerical magnitude does not represent sentiment or semantic similarity. An embedding layer converts each index into a dense vector whose coordinates can be learned from the rating-prediction task [4].

Pretrained fastText vectors offer another initialization strategy. They provide word representations learned from a larger language corpus. In the experiment, Portuguese vectors are loaded from a compressed text file and kept frozen. This differs from learning the embedding directly from the Olist reviews. Although fastText uses subword information when creating its representations, this implementation loads a fixed word-vector table and does not generate vectors dynamically for unseen words.

B. Convolutional Neural Networks for Text

A one-dimensional convolution applies the same learned filter to successive groups of word vectors. With a kernel width of three, each filter examines a local three-token window. The filter can learn patterns associated with positive or negative experiences, although the meaning of individual filters is not directly established by the experiment. Global max pooling retains the strongest activation of each filter anywhere in the review, and a final classifier converts these features into rating probabilities.

C. Ordered Review Scores

The ratings represent ordered categories rather than unrelated labels. Nevertheless, ordinary multiclass cross-entropy treats the correct class as the target without explicitly encoding the distance between ratings. An ordinal objective can instead express a rating through cumulative questions, such as whether it is greater than one, two, three, or four stars [6]. Both approaches are evaluated because classification quality and rating-distance error don't improve together.

IV. Dataset and Data Preparation

A. Dataset Source and Input Variables

The data come from the Brazilian E-Commerce Public Dataset by Olist, available on Kaggle [1]. This study uses the order-review table. Other tables, including customers, sellers, geolocation, payments, products, and order items, are not joined because the intended prediction is based on the written review itself. Including additional transaction attributes would define a different experiment.

Field	Role
review_score	Target, from 1 to 5
review_comment_title	Optional text input
review_comment_message	Required comment body
review_id / order_id	Identifiers for cleaning
Review timestamps	Excluded from predictors

Table 1. Roles of the review-table fields.

The original review table contains 99,224 rows and seven columns. There are 40,977 non-null comment bodies and 11,568 non-null titles. Consequently, many transactions contain a score without a written comment. Such observations cannot support the text-only prediction task in its present form.

V. Exploratory Data Analysis

A. Data Quality Assessment

The preparation notebook checks duplicated review identifiers and repeated order-comment combinations. There are 814 occurrences beyond the first occurrence of each review identifier. Keeping the first occurrence reduces the table to 98,410 rows. The cleaning procedure also removes repeated order-comment combinations, excludes missing or blank comment bodies, and replaces missing titles with empty strings. The final modeling dataset contains 40,634 reviews. A review with only a title is excluded because the procedure requires a nonempty body.

B. Target Class Distribution

Score	Count	Percentage
1	8,640	21.26%
2	2,118	5.21%
3	3,528	8.68%
4	5,935	14.61%
5	20,413	50.24%

Table 2. Score distribution after cleaning.

The distribution shows a concentration at the endpoints. Five-star reviews account for approximately half of the dataset, while two-star reviews form the smallest class. A model that favors the most common ratings can therefore obtain a moderate overall accuracy while failing to recognize intermediate scores. Class-level recall and macro-F1 are necessary to reveal this behavior.

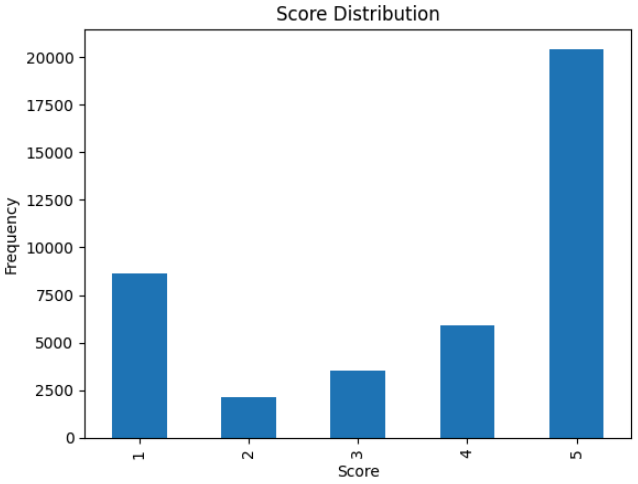


Figure 1. Distribution of the 40,634 cleaned reviews.

C. Text and Label Ambiguity

The previously inspected errors include negative descriptions labeled with two stars that the model predicts as one star. Statements about undelivered products, missing items, or poor quality indicate dissatisfaction, but may not specify whether the author intended one or two stars. Similar ambiguity can occur between four and five stars. These examples suggest overlap in the language used across adjacent ratings; they do not establish that the labels are incorrect or that a performance ceiling has been reached.

VI. Data Preprocessing

A. Text Construction and Label Encoding

The optional title and the comment body are concatenated into a single string. TextVectorization lowercases the text and removes punctuation using its default standardization. Whitespace separates tokens [2]. The vocabulary is adapted using the training texts only and is limited to 15,000 entries. Accented Portuguese words remain part of the vocabulary. For the CNNs, reviews are represented as integer sequences of length 150, with padding or truncation as necessary.

Scores are encoded internally as 0–4 and converted back to 1–5 for interpretation.

B. Training, Validation, and Test Partition

Two stratified splits with random_state=42 produce 26,005 training reviews, 6,502 validation reviews, and 8,127 test reviews, corresponding approximately to 64%, 16%, and 20% of the data. Stratification preserves the rating distribution. The training subset is used to adapt the vocabulary and fit model parameters. The validation subset supports early stopping and model comparison. The test subset is used for the final evaluation of the selected Embedding CNN.

The split is performed at the review-row level. It does not explicitly group reviews by customer or identical text, so independence across these groups is not guaranteed. Future evaluation could use grouped or chronological partitions to investigate generalization to different customers or later transactions.

C. Class Imbalance Treatment

The CNN classification experiments use the square root of balanced class weights, which increases the contribution of less frequent classes without applying the full inverse-frequency correction. For class k with n_k training observations and N training reviews, the weight is given by

$$w_k = \sqrt{[N / (5n_k)]}$$

The approximate weights for scores one through five are 0.9699, 1.9585, 1.5177, 1.1702, and 0.6310. These weights are computed from the training labels. No synthetic reviews are introduced. The TF-IDF baseline uses its original unweighted configuration, so the comparison reflects complete experimental configurations rather than an isolated comparison of representations.

VII. Classification Methodology

A. TF-IDF and Multinomial Logistic Regression

The first model transforms each review into a TF-IDF vector. Under the Keras TextVectorization convention used here, the inverse document frequency for term t can be expressed as follows, where N is the number of training documents and df(t) is the number containing that term [2].

$$\text{tf}(t, d) = \frac{\textit{number of times t appears in a document}}{\textit{total number of terms in d}}$$

$$\text{idf}(t) = \log[\, N / (1 + \text{df})]$$

$$\text{tf - idf}(t,d) = \text{tf}(t, d) \cdot \text{idf}(t)$$

A dense layer maps this vector directly to five logits, followed by softmax. With no hidden layer, this architecture implements multinomial logistic regression. It contains 75,005 parameters. Its computational simplicity makes it a useful baseline, although the bag-of-words representation does not preserve token order.

$$p_k = \exp(z_k) / \Sigma_{j=1}^{5} \exp(z_j),$$

$$z = Wx + b$$

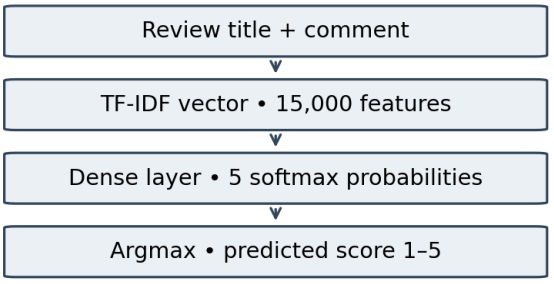


Figure 2. TF-IDF baseline architecture.

B. Fast Text CNN

The second model uses the same integer vocabulary to retrieve pretrained 300-dimensional Portuguese fastText vectors [2]. The compressed vector file is read sequentially, retaining only the vectors needed for the training vocabulary. A total of 11,620 of 14,998 non-special vocabulary entries are matched, corresponding to 77.5% coverage. Unmatched entries receive small random vectors, and the padding vector is initialized to zero. The embedding remains frozen during training.

The embedding output passes through a Conv1D layer with 128 filters, kernel size three, and ReLU activation. Global max pooling reduces the sequence to 128 features, dropout uses a rate of 0.3, and a dense softmax layer returns five probabilities. The model contains 4,615,973 parameters, of which 115,973 are trainable. Freezing the embeddings reduces the number of updated parameters, but also prevents unmatched random vectors from adapting to the task.

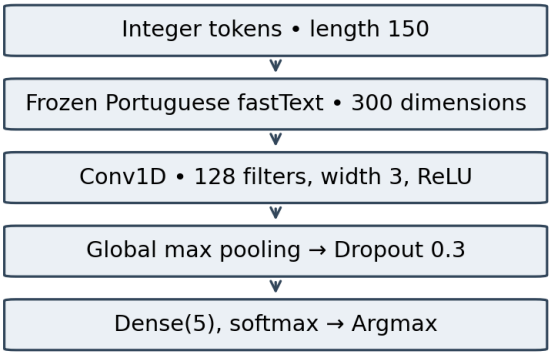


Figure 3. CNN with frozen fastText word vectors.

C. Embedding CNN

The third model learns its embedding table directly from the training reviews. Each token receives a 128-dimensional vector. The convolution, pooling, dropout, and output layers

follow the same design as the fastText model. In this setting, the embedding coordinates are updated together with the convolution and classifier, allowing the representation to reflect the rating-prediction objective.

$$e_i = E[\text{token}_i]$$
$$h_{i,j} = \text{ReLU}(\langle F_j, E_{i:i+2} \rangle + b_j)$$
$$g_j = \max_i h_{i,j}$$

Here, E is the embedding matrix, F_j is the j-th convolution filter, and g_j is its pooled activation. The model contains 1,969,925 trainable parameters. Most parameters belong to the 15,000-by-128 embedding table. The convolution contains 49,280 parameters, and the final dense layer contains 645 parameters.

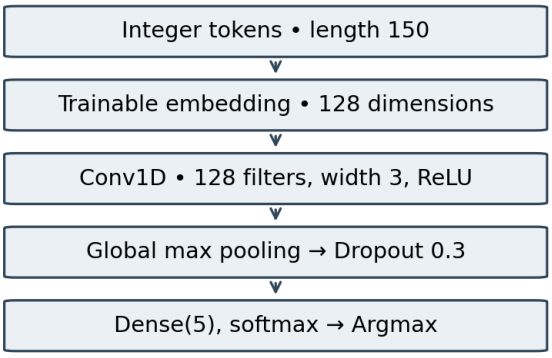


Figure 4. Selected CNN with learned word embeddings.

D. Ordinal Embedding CNN

The fourth model uses a fresh learned-embedding CNN with a five-class softmax output. During training, a fixed, nontrainable transformation converts the five probabilities into four cumulative probabilities. For a rating r on the 1–5 scale, the threshold targets and probabilities are defined by

$$t_k = 1[r > k], \quad q_k = \Sigma_{j=k+1}^{5} p_j, \quad k = 1,\ldots,4$$

For example, a three-star review has threshold targets [1, 1, 0, 0]. Binary cross-entropy compares the cumulative probabilities with these targets. The same class-derived sample weights are applied. Because each cumulative probability is a sum of the softmax probabilities, their ordering is consistent by construction. At inference, the shared five-class predictor returns star probabilities and the most probable star is selected.

$$L_{\text{ordinal}} = -(1/4) \Sigma_{k=1}^{4} s_k$$
$$s_k = t_k \ln(q_k) + (1-t_k) \ln(1-q_k)$$

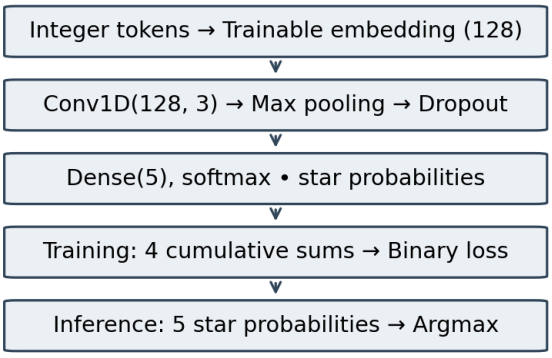


Figure 5. Ordinal training and star-score inference.

E. Optimization and Early Stopping

The models are trained using Adam, batches of 64 reviews, and a maximum of 50 epochs. The ordinary classifiers use sparse categorical cross-entropy. For the class-weighted CNNs, the contribution of each training example is multiplied by its class weight.

$$L = -(1/N) \Sigma_{i=1}^{N} w_{\gamma i} \ln(p_{i,\gamma i})$$

Early stopping monitors validation loss with a patience of three epochs and restores the weights from the best epoch. Thus, 50 epochs is an upper limit rather than a required duration. The selected Embedding CNN reaches its lowest validation loss at epoch two and stops after epoch five. The ordinal model also restores its second-epoch weights.

F. Regression Experiments

Two regression experiments were added to investigate whether predicting a numerical rating directly could reduce the distance between predicted and observed scores. Both use the existing training and validation partitions and retain labels on the internal 0–4 scale. Predictions are converted back to stars by adding one. These experiments extend the comparison while keeping the text preparation consistent with the corresponding classification models.

The Embedding CNN regressor retains the vocabulary limit of 15,000 tokens, sequence length of 150, trainable 128-dimensional embeddings, Conv1D layer with 128 filters of width three, global max pooling, and dropout of 0.3. Its five-class softmax layer is replaced by a single linear output. The resulting model contains 1,969,409 trainable parameters. The regression experiment uses mean absolute error as the training loss, Adam, and batches of 64. Class weights are not applied. Early stopping monitors unrounded, unclipped validation MAE with patience three and restores the best weights. The best epoch is five, with validation MAE approximately 0.6195, and training stops after epoch eight.

The TF-IDF Ridge regressor reuses the fitted Keras TF-IDF vectorizer. Features are generated in batches and converted to sparse matrices to limit memory consumption. Ridge fits a linear prediction with an L2 penalty on its

coefficients, using alpha = 1.0 and the lsqr solver [6]. It is trained on the training subset without class weights; no regularization search is performed. Unlike the CNN, this estimator does not use Keras epochs or early stopping.

$$\textit{CNN regression objective}: L = (1/N)\,\Sigma_i\,|y_i - f(x_i)|$$

$$\textit{Ridge objective}: \Sigma_i\,(y_i - x_i^T\beta - b)^2 + \alpha\,\Sigma_j\,\beta_j^2$$

The regression CNN therefore minimizes absolute error, whereas Ridge minimizes squared error plus regularization. They should not both be interpreted as MSE-trained models. Differences from the classification CNN also include the output layer, loss, class weighting, and stopping criterion, so the comparison concerns the complete tested configurations.

G. Evaluation Metrics

Accuracy measures the proportion of reviews assigned the exact observed rating. Precision indicates how many predictions of a class are correct, whereas recall indicates how many actual examples of that class are recovered. F1 combines precision and recall through their harmonic mean.

$$\textit{Accuracy} = (1/N)\,\Sigma_{i=1}^{N}\,1[\hat{y}_i = y_i]$$

$$\textit{Precision}_k = TP_k / (TP_k + FP_k)$$

$$\textit{Recall}_k = TP_k / (TP_k + FN_k)$$

$$F1_k = 2TP_k / (2TP_k + FP_k + FN_k)$$

$$\textit{Macro-F1} = (1/5)\,\Sigma_{k=1}^{5}\,F1_k$$

Macro-F1 gives equal importance to each score, regardless of its frequency. It is therefore useful for identifying weak intermediate-class performance that can be hidden by the majority classes. MAE measures rating distance in stars, while within-one-star accuracy counts predictions whose absolute error is at most one.

$$MAE = (1/N)\,\Sigma_{i=1}^{N}\,|y_i - \hat{y}_i|$$

$$W_1 = (1/N)\,\Sigma_{i=1}^{N}\,1[|y_i - \hat{y}_i| \le 1]$$

A low MAE and a modest macro-F1 can coexist. Predicting one star for a two-star review is an incorrect class prediction but contributes only one star of absolute error. Correct predictions for the frequent endpoint classes further reduce overall MAE. Consequently, MAE alone does not demonstrate reliable recognition of every rating.

For regression, the predicted rating is clipped to the valid interval [1, 5]. MAE and root mean squared error (RMSE) are computed on these decimal predictions. RMSE is the square root of the average squared error and gives larger errors greater influence. Classification metrics are then calculated after rounding to the nearest integer, with half-star ties rounded upward.

$$\textit{RMSE} = \sqrt{[(1/N)\,\Sigma_i\,(y_i - \hat{y}_i)^2]}$$

$$\textit{Rounded prediction} = \text{floor}(\text{clip}(f(x) + 1, 1, 5) + 0.5)$$

Rounded MAE is reported separately to compare integer outputs with the classification models. Clipping is applied only during regression evaluation, which explains why the reported decimal MAE may be lower than the validation MAE recorded during training.

H. Baseline Model Results

The TF-IDF baseline achieves a validation accuracy of 69.06%, a macro-F1 of approximately 0.41, and an MAE of 0.4882 stars. Its one- and five-star F1 scores are approximately 0.76 and 0.85, respectively. In contrast, the two-, three-, and four-star F1 scores are approximately 0.10, 0.16, and 0.19. These results establish that useful rating-related information is present in the text, but the baseline favors the endpoint classes.

I. CNN Model Improvement and Comparison

Model	Accuracy	Macro-F1	MAE
TF-IDF	0.6906	≈0.41	0.4882
fastText CNN	0.6912	0.4266	0.4843
Embedding CNN	0.6838	0.4511	0.4803
Ordinal CNN	0.6916	0.4306	0.4780

Table 3. Validation comparison using argmax predictions.

The frozen fastText model slightly improves the baseline macro-F1 and MAE, but its two-star recall remains 0.0531. Pretrained word vectors therefore do not automatically resolve the distinction between closely related rating categories. Partial vocabulary coverage and the frozen representation are possible contributing factors, although the present experiments do not isolate their individual effects.

The learned Embedding CNN achieves the highest macro-F1 among the four configurations under the common argmax rule. Its accuracy is lower than the other models, but it improves the balance between exact category recognition and rating distance. Its two-star recall rises to 0.1298, compared with 0.0531 for fastText. This remains a weak absolute result, but represents greater recognition of the rare class.

Model	2-star recall	Within 1 star
Fast_Text_CNN	0.0531	0.8834
Embedding CNN	0.1298	0.8902
Ordinal CNN	0.0059	0.8867

Table 4. Complementary validation metrics.

The ordinal model obtains the lowest validation MAE, 0.4780 stars, and the highest accuracy, 0.6916, among these configurations. However, its two-star recall falls to 0.0059, meaning that only approximately two of the 339 actual two-star reviews are correctly identified. The ordinal objective improves average rating distance slightly while reducing intermediate-class recognition. It is therefore not selected for the final application.

J. Training Behavior

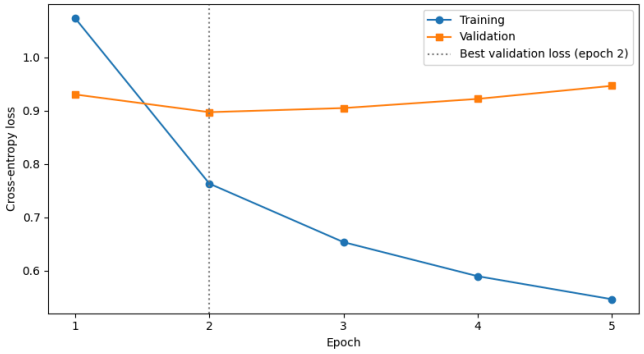


Figure 6. Loss history of the selected Embedding CNN.

The validation loss from the Embedding CNN model decreases from 0.9029 to 0.8745 during the first two epochs and then increases, reaching 1.0340 at epoch five. Meanwhile, training accuracy rises from 0.6475 to 0.8168. This pattern indicates overfitting after the best validation epoch. Figure 6 shows weighted training loss and unweighted validation loss, so their absolute levels are not directly comparable; their trends support the early-stopping decision.

K. Prediction Rule Analysis

The most probable star is the category with the largest individual probability. The median star is the first category whose cumulative probability reaches 0.5. They are alternative decision rules applied to the same model outputs.

$$\hat{y}\text{mode} = \text{argmax}_k\, p_k$$

$$\hat{y}\text{median} = \min\{k : \Sigma_{j=1}^{k}\, p_j \geq 0.5\}$$

For the selected CNN, the median rule raises validation macro-F1 from 0.4511 to 0.4584 and two-star recall from 0.1298 to 0.3805. However, accuracy decreases from 0.6838 to 0.6203 and MAE increases from 0.4803 to 0.5043. The argmax rule is retained for the final evaluation. Although a distributional median minimizes absolute loss under that predicted distribution, estimated probabilities can be imperfect, particularly after class weighting, so empirical MAE need not improve.

L. Regression Results and Comparison

The MAE-trained Embedding CNN regressor obtains a clipped decimal validation MAE of 0.5963 stars and RMSE of 0.8919 stars. After rounding, its MAE is 0.5584, accuracy is approximately 0.56, and macro-F1 is approximately 0.42. TF-IDF with Ridge obtains a decimal MAE of 0.7954 and RMSE of 1.1609. Its rounded MAE is 0.7795, accuracy is approximately 0.46, and macro-F1 is approximately 0.36. Table 5 summarizes these results on the same 6,502 validation reviews.

Model	**Decimal MAE**	**RMSE**	**Rounded MAE**
CNN classifier	N/A	N/A	0.4803
CNN regression (MAE loss)	0.5963	0.8919	0.5584
TF-IDF Ridge	0.7954	1.1609	0.7795

Table 5. Validation regression results and classifier reference.

The classifier reference in Table 5 is its original integer-prediction MAE. Both regression configurations produce higher integer-prediction MAE and lower accuracy and macro-F1 than the selected classifier. The regression CNN recovers approximately 39% of two-star reviews, but its precision for that class is only 14%. Its one-star recall is approximately 44%, compared with approximately 86% for the classifier. Ridge obtains approximately 22% two-star recall and 42% one-star recall. Improved recall for an intermediate class therefore does not translate into a better overall result.

The observed results support retaining the classification Embedding CNN. They do not establish that regression is generally unsuitable for review ratings. Ridge was evaluated at a single alpha value, and neither regression experiment represents an exhaustive model search. The CNN uses absolute error, which targets a conditional median in the ideal population setting, while squared-error Ridge targets a regularized approximation to the conditional mean. Performance also depends on text representation, optimization, and model capacity.

These regression experiments were added after the classification model had already been evaluated on the test subset. Their reported comparison uses validation results only; neither regression notebook contains a final test evaluation. The existing classifier test results are retained as previously obtained rather than presented as a new test of all six configurations.

M. Final Model Selection and Test Evaluation

The Embedding CNN is selected as the preferred model for the present objective because it provides the strongest validation macro-F1 among the four argmax configurations, the highest within-one-star accuracy among the CNNs, and an MAE close to the best observed result. Its validation

MAE is only 0.0023 stars higher than that of the ordinal model, while its two-star recall is substantially greater. This is a selection based on complementary criteria rather than a claim that the model leads every metric.

After model selection, the selected Embedding CNN is evaluated on 8,127 test reviews using the same argmax rule. The test MAE is 0.4924 stars, compared with 0.4803 on validation, representing an increase of approximately 0.0121 stars. Exact test accuracy is 67.82%, while macro-F1 and weighted F1 are approximately 0.45 and 0.65, respectively. The F1 values are reported to the precision available in the saved classification report.

Score	Precision	Recall	F1	Support
1	.68	.86	.76	1,728
2	.20	.13	.16	423
3	.28	.24	.26	706
4	.35	.19	.25	1,187
5	.81	.88	.84	4,083

Table 6. Final Embedding CNN test classification report.

The test results preserve the pattern observed during validation. The model recognizes one- and five-star reviews relatively well, while recall for scores two, three, and four remains low. An MAE below half a star means that the average absolute rating error is approximately 0.49 stars across all test examples. It does not mean that every prediction is within half a star.

For perspective, always predicting five stars on this test distribution would yield approximately 50.24% accuracy and an MAE of 1.3264 stars. The selected model improves substantially over that constant prediction. Nevertheless, the low intermediate-class F1 scores limit its suitability for decisions requiring accurate distinctions between neighboring ratings.

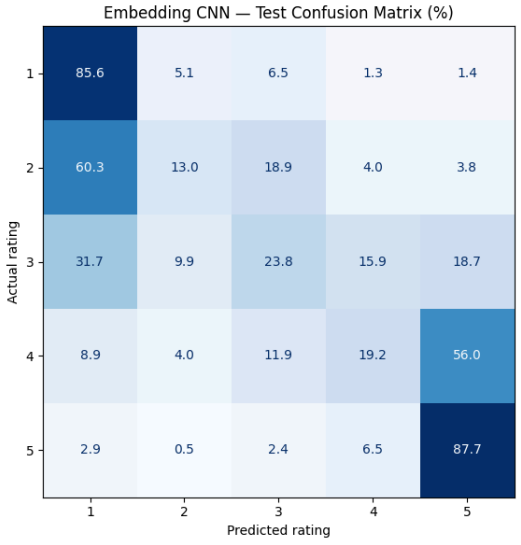


Figure 7. Row-normalized confusion matrix of the selected Embedding CNN on the test set. Each cell represents the percentage of reviews with a given actual rating assigned to each predicted rating.

The row-normalized confusion matrix shows that the Embedding CNN recognizes endpoint ratings more effectively than intermediate ratings. It correctly identifies 85.6% of one-star reviews and 87.7% of five-star reviews. In contrast, recall is 13.0% for two-star reviews, 23.8% for three-star reviews, and 19.2% for four-star reviews. The most frequent confusions involve two-star reviews predicted as one star (60.3%) and four-star reviews predicted as five stars (56.0%). This pattern suggests that the model captures broad positive and negative sentiment more successfully than distinctions between adjacent rating levels.

Overall, exact test accuracy is 67.82%, while 88.46% of predictions fall within one star of the observed rating. Therefore, 20.64% of all predictions are incorrect by exactly one star, and 11.54% differ by two or more stars. These results help explain how relatively low MAE can coexist with modest macro-F1: neighboring-rating errors receive no credit in exact classification metrics but contribute a smaller rating-distance penalty than larger errors.

Nevertheless, substantial errors remain. For example, 31.7% of three-star reviews are predicted as one star. The strong performance on the frequent endpoint classes also contributes to the aggregate results, so the within-one-star rate should not be interpreted as equally reliable performance across all ratings. The confusion matrix supports the selected model's usefulness for approximate rating estimation while confirming its limitations for applications requiring precise recognition of intermediate scores.

N. Model Reuse and User Interface

The trained model is saved in the Keras format with its text vectorization and learned parameters. A simple interface accepts a Portuguese title and comment, combines them into the same input format, and displays the most probable rating with the five output probabilities. Keeping preprocessing inside the saved model helps preserve the vocabulary mapping during inference. The displayed probabilities are model scores and are not established as calibrated confidence estimates.

VIII. Conclusion

This project compares four classification approaches and two additional regression configurations to Portuguese review-score prediction using a shared data preparation procedure and stratified partition. The TF-IDF baseline captures useful sentiment information but performs poorly on intermediate ratings. Frozen fastText vectors provide a modest change in performance, while learning task-specific embeddings yields a better macro-F1 under the common argmax prediction rule. The ordinal objective slightly reduces MAE but almost eliminates correct recognition of two-star reviews. The MAE-trained regression CNN and TF-IDF Ridge regressor both underperform the classification CNN on rounded validation MAE, accuracy, and macro-F1.

The Embedding CNN is selected because it offers the preferred balance across the evaluation criteria. Its test MAE

of 0.4924 stars supports its use as an approximate rating estimator, and its test results are broadly consistent with validation. At the same time, a macro-F1 of approximately 0.45 and low intermediate-class recall show that fine-grained rating classification remains difficult. The reported results should therefore be interpreted as useful progress on approximate prediction, with clear class-specific limitations.

The comparison is based on individual saved training runs rather than repeated-seed estimates, so small metric differences do not establish statistical superiority. Other limitations include review-level splitting, exclusion of reviews without comment bodies, truncation to 150 tokens, and possible ambiguity between adjacent scores. Future work could investigate grouped evaluation, contextual Portuguese language models, probability calibration, and repeated runs. Further model development should use validation data, with a fresh held-out evaluation if the current test results influence new design choices.

References

1. Brazilian E-Commerce Public Dataset by OLIST. (2021, October 1). https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

2. Word vectors for 157 languages · fastText. (n.d.). https://fasttext.cc/docs/en/crawl-vectors.html

3. Team, K. (n.d.). Keras documentation: TextVectorization layer. https://keras.io/api/layers/preprocessing_layers/text/text_vectorization/

4. Team, K. (n.d.). Keras documentation: Embedding layer. https://keras.io/api/layers/core_layers/embedding/

5. Frank, E., & Hall, M. (2001). *A simple approach to ordinal classification*. University of Waikato. https://ml.cms.waikato.ac.nz/publications/2001/ordinal_tech_report.pdf

6. Scikit-learn developers. (n.d.). *Ridge*. Scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html